Activity Selection

Greedy Strategy

Core Strategy:
- Sort activities by finishing time.
- Always pick the earliest finishing compatible activity.

C++ Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Activity {
    int start, end;
};

bool cmp(Activity a, Activity b) {
    return a.end < b.end;
}

int main() {
    int n;
    cin >> n;

    vector<Activity> a(n);

    for (int i = 0; i < n; i++)
        cin >> a[i].start >> a[i].end;

    sort(a.begin(), a.end(), cmp);

    cout << "Selected Activities:\n";
    cout << "(" << a[0].start << "," << a[0].end << ")\n";

    int lastEnd = a[0].end;
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (a[i].start >= lastEnd) {
            cout << "(" << a[i].start << "," << a[i].end << ")\n";
            lastEnd = a[i].end;
            count++;
        }
    }

    cout << "\nMaximum Activities = " << count;

    return 0;
}
```

Complexity Analysis

Time Complexity:
- **Sorting:** $\mathcal{O}(N \log N)$ where N is the number of activities.
- **Greedy Pass:** $\mathcal{O}(N)$ to iterate and filter out valid activities.
- **Total Time Complexity:** $\mathcal{O}(N \log N)$.

Space Complexity:
- $\mathcal{O}(N)$ auxiliary memory to store the structure of N activities.

Quick Reference

Input Format:
n (Number of activities)
start_1 end_1
start_2 end_2
...

Output Format:
Interval representation of chosen sequences
Total count of successfully scheduled tasks

Fractional Knapsack

Greedy Strategy

Core Strategy:
- Sort by value/weight ratio.
- Take the highest ratio first.

C++ Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Item {
    int value, weight;
};

bool cmp(Item a, Item b) {
    return (double)a.value / a.weight > (double)b.value / b.weight;
}

int main() {
    int n, W;
    cin >> n >> W;

    vector<Item> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i].value >> a[i].weight;

    sort(a.begin(), a.end(), cmp);

    double profit = 0;
    for (int i = 0; i < n; i++) {
        if (W >= a[i].weight) {
            profit += a[i].value;
            W -= a[i].weight;
        } else {
            profit += (double)a[i].value / a[i].weight * W;
            break;
        }
    }

    cout << profit;

    return 0;
}
```

Complexity Analysis

Time Complexity:
- **Sorting Ratio:** $\mathcal{O}(N \log N)$ where N is the item count.
- **Greedy Loop:** $\mathcal{O}(N)$ to traverse items and fill capacity.
- **Total Time Complexity:** $\mathcal{O}(N \log N)$.

Space Complexity:
- $\mathcal{O}(N)$ memory limit allocation to preserve vector items.

Quick Reference

Input Format:
n (Items) W (Max Knapsack capacity)
value_1 weight_1
value_2 weight_2
...

Output Format:
Decimal maximum possible accumulated value

Longest Common Subsequence (LCS)

Dynamic Programming

Core Strategy:
- If characters match: $dp[i][j] = dp[i-1][j-1] + 1$.
- Else: $dp[i][j] = \max(\text{top}, \text{left})$.

C++ Implementation

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    string a, b;
    cin >> a >> b;

    int n = a.size();
    int m = b.size();

    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    cout << "Length = " << dp[n][m] << endl;

    string lcs = "";
    int i = n, j = m;

    while (i > 0 && j > 0) {
        if (a[i - 1] == b[j - 1]) {
            lcs += a[i - 1];
            i--;
            j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            i--;
        } else {
            j--;
        }
    }

    reverse(lcs.begin(), lcs.end());

    cout << "LCS = " << lcs;

    return 0;
}
```

Complexity Analysis

Time Complexity:
- **Matrix Fill:** $\mathcal{O}(N \times M)$ nested grid calculations.
- **Backtracking:** $\mathcal{O}(N + M)$ to extract matching path.
- **Total Time Complexity:** $\mathcal{O}(N \times M)$ where N, M are lengths.

Space Complexity:
- $\mathcal{O}(N \times M)$ memory footprint for the 2D DP table.

Quick Reference

Input Format:
First Sequence (string A)
Second Sequence (string B)

Output Format:

Length of LCS ($dp[n][m]$)
The exact matching sequence string

0/1 Knapsack

Dynamic Programming

Core Strategy:
- Take or Don't Take.
- Choose maximum profit.

C++ Implementation

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n, W;
    cin >> n >> W;

    vector<int> weight(n + 1), value(n + 1);

    for (int i = 1; i <= n; i++)
        cin >> weight[i];

    for (int i = 1; i <= n; i++)
        cin >> value[i];

    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= W; j++) {
            if (weight[i] <= j)
                dp[i][j] = max(dp[i - 1][j], value[i] + dp[i - 1][j - weight[i]]);
            else
                dp[i][j] = dp[i - 1][j];
        }
    }

    cout << dp[n][W];

    return 0;
}
```

Complexity Analysis

Time Complexity:
- **DP Fill:** $\mathcal{O}(N \times W)$ lookup computations.
- **Total Time Complexity:** $\mathcal{O}(N \times W)$ where N is total items and W is total knapsack storage limit.

Space Complexity:
- $\mathcal{O}(N \times W)$ size to support 2D states table.

Quick Reference

Input Format:
n (Items count) W (Capacity of knapsack)
$w_1 \quad w_2 \quad \dots \quad w_n$ (Weights array)
$v_1 \quad v_2 \quad \dots \quad v_n$ (Values array)

Output Format:

Maximum possible profit value ($dp[n][W]$)

Rod Cutting

Dynamic Programming

Core Strategy:
- Try every first cut.
- Choose maximum obtainable profit.

C++ Implementation

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> price(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> price[i];

    vector<int> dp(n + 1, 0);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            dp[i] = max(dp[i], price[j] + dp[i - j]);
        }
    }

    cout << dp[n];

    return 0;
}
```

Complexity Analysis

Time Complexity:
- **Nested Loops:** Outer loop i up to N, inner loop j up to i. Total operations $\approx \sum_{i=1}^{N} i = \frac{N(N+1)}{2}$.
- **Total Time Complexity:** $\mathcal{O}(N^2)$ where N is rod length.

Space Complexity:
- $\mathcal{O}(N)$ space storing the 1D profit tracking lookup array.

Quick Reference

Input Format:
n (Total length of the rod)
$p_1 \quad p_2 \quad \dots \quad p_n$ (Individual price value cuts)

Output Format:

Maximum value achievable by cutting the rod